Traceable Question Answering over Enterprise Research Documents

A privately deployed RAG system and a human-AI task evaluation

Lintong Han, Mingchu Ge, Xianglian Zeng, Liangqiang Chen,
Mingyue Cheng, Xu Wang, Xingyong Gao

Application report, 2026. Adapted from an internal Chinese write-up. Not peer reviewed.

Abstract

Research and development departments accumulate large volumes of project documents: task statements, progress summaries, technical reports, test records and standards, stored as Word, PDF, Excel and scanned files and revised many times. Keyword search over such material finds words but rarely the right version of the right passage, and it cannot show where an answer came from. We built a retrieval-augmented generation (RAG) question-answering system that runs entirely inside the company network. Its pipeline recovers document structure, splits content into meaningful segments, retrieves with both keyword and vector search, re-ranks candidates, generates answers only from the selected evidence, and links every answer back to the file, page and paragraph it came from. We evaluated it on a four-step task chain over 50 research-project documents, comparing a person using the system, the system alone, and a person using conventional tools. The person using the system completed all tasks with 97.75% accuracy in an average of 141 s per task chain; the system alone was faster but less accurate (81%), and conventional work was accurate on what it finished but completed only half of the task chains within the time limit, taking 498 s on average. The results support using such a system as an assistant that keeps a person in the loop, rather than as a replacement.

1 Problem

Managers of research projects need to answer questions such as: which project does this summary describe, where are its targets stated, what did the latest phase report conclude. Three properties of real document collections make this hard.

- **Heterogeneous formats.** Text, tables, figures and scanned pages coexist, often in the same file.
- **Frequent revision.** The same document exists in several versions, and the newest is not always the one keyword search finds first.
- **Accountability.** An answer is only useful if the reader can check it: which file, which version, which page.

RAG systems combine retrieval with generation to ground answers in external documents [1]. Work on document layout, tables and charts [2, 3, 4] and on retrieval [5, 6, 7] provides the building blocks. What was missing for our setting was an end-to-end system that handles all three properties at once, runs on local hardware, and was tested on the tasks people in the department actually do.

2 System design

2.1 Building the knowledge base (offline)

Ingestion and structure recovery. Each document receives an identifier and metadata: version, source, access level and time of ingestion. Text documents keep their headings, paragraphs and page numbers. Scanned pages go through OCR, and a document-vision model restores the link between text and its position on the page. Tables keep their header paths and cell indices; figures keep their numbers and captions. Every document becomes a structure tree.

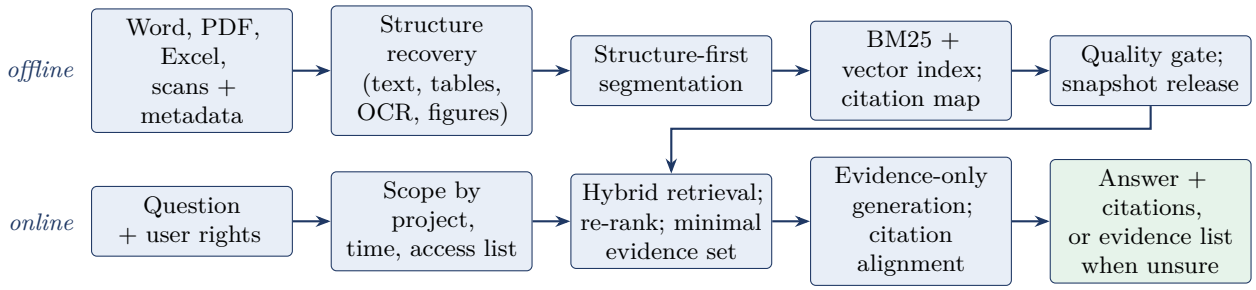


Figure 1: System overview. Offline, documents are parsed, segmented, indexed and checked before a snapshot of the knowledge base is released. Online, each question is scoped to what the user may see, answered only from retrieved evidence, and linked back to its sources. Everything runs inside the company network.

Segmentation. Splitting follows structure first: headings, paragraphs, clause numbers and table boundaries. Only segments that are too long or that cross a topic change are split further with an overlapping window. Each segment carries its source path, page and position, plus a short description used in retrieval.

Indexing and the citation map. Segments are embedded with a Chinese BGE model and stored in a FAISS vector index alongside a BM25 keyword index [5, 8]; the system also supports Qdrant as a vector backend. Retrieval works in two stages, first selecting candidate documents and then segments within them. A citation map links every segment to its file, version, page, paragraph or table path, including aliases for clauses and table paths and offsets across versions, so that any answer can be traced back even after documents are revised.

Quality gate. Before a knowledge-base snapshot is released, the system checks OCR confidence, redundancy, the share of segments that can be cited, coverage and index size. Segments with low OCR confidence stay searchable but are marked as not citable. If overall quality falls below thresholds the release stops and a report is written. All build settings are kept in a configuration file, so a snapshot can be reproduced or rolled back. If the serving side detects a missing or mismatched snapshot, it falls back to read-only search.

2.2 Answering a question (online)

Scoping. The question is first restricted by metadata (project number, contract, time window, document type) and by the user's access list, so retrieval never touches documents the user may not see. The query is then normalized (entities, numbers, synonyms).

Retrieval and evidence selection. Keyword and vector retrieval run in parallel at document and segment level. Candidates are de-duplicated and older versions are ranked below the current one. A lightweight cross-encoder re-ranks the rest, and the system keeps a small set of segments that together cover the question. For tables, the header path, units and time basis are kept with the numbers.

Evidence-only generation. The selected segments are inserted into a fixed prompt template with their version and location, and a locally deployed LLM is instructed to answer only from them, with conservative decoding settings. Each answer sentence is then aligned with its supporting segment and mapped through the citation map to file, page and paragraph. The output is structured: answer text, citation list, evidence set and run metadata.

Saying less when unsure. If evidence is insufficient, the system returns the evidence it found and what is missing instead of an answer. If versions conflict and cannot be resolved automatically, it shows both with their dates. If the user lacks access, it stops before generation. Every such event is logged.

Extensions. The system leaves room for a light graph of relations between segments, for multi-turn tasks coordinated by a small agent layer, and for integration with the project-management platform. These are interfaces, not evaluated features.

3 Evaluation

3.1 Task chain and conditions

From publicly available material from 2023–2024, we built 50 research-project documents in the format of internal project task statements, with reference answers. Each of the 50 test items is a chain of four steps (200 steps in total):

1. **Identify:** given a project summary without its name, name the project and its file.
2. **Locate:** find the passage that states its research content and phase results, down to page or paragraph.
3. **Extract:** pull out key indicators, research goals and results.
4. **Summarize:** write a 150–200 word summary with at least one source citation.

Three conditions were compared: a person working with the system (collaborative), the system alone, and a person with conventional tools (Excel, Word, keyword search). Each step had a time limit, nine minutes for the whole chain.

3.2 Metrics and judging

Completion rate is the share of tasks with a valid submission within the time limit. Accuracy is the share of valid submissions judged correct, so its denominator is the number of valid submissions. Time is averaged over valid submissions. Correctness was judged by two independent reviewers against published rules (exact project name; page-level overlap of at least 0.5 with the reference location, with one page of tolerance; key-phrase agreement for extraction; no factual errors and adequate coverage for summaries), with a third reviewer resolving disagreements.

3.3 Results

The system ran on a local server with a single NVIDIA RTX 4070 Ti GPU and 64 GB of system memory, using LangChain and Xinference to serve locally deployed Qwen and DeepSeek models. All times below were measured on this machine; faster hardware would mainly shorten the system's own response time.

What the numbers show. The collaborative condition was the most balanced: every chain completed, accuracy near 98%, and less than a third of the conventional time. The system alone was fastest but made more errors, especially where versions differed or context had to be carried from one step to the next; with a person checking its output, accuracy rose to 97.75%. Conventional work was accurate on what it finished, but under time pressure half the chains were not finished, and summarizing took far longer than any other step.

Table 1: Completion rate, accuracy (of valid submissions) and mean time per task; 50 tasks per step.

	Collaborative			System only			Conventional			Limit
Step	Comp.	Acc.	Time	Comp.	Acc.	Time	Comp.	Acc.	Time	
Identify	100%	98%	56 s	92%	81%	34 s	74%	87%	118 s	2 min
Locate	100%	97%	25 s	88%	80%	20 s	74%	96%	35 s	1 min
Extract	100%	98%	45 s	86%	81%	31 s	74%	97%	56 s	1 min
Summarize	100%	98%	15 s	86%	80%	15 s	50%	98%	289 s	5 min
Full chain	**100%**	**97.75%**	141 s	86%	81%	**100 s**	50%	95%	498 s	9 min

A caution on reading accuracy. Because accuracy is computed over valid submissions, the conventional condition's 95% is measured on only the half of the chains that were finished, which are likely the easier ones. Completion and accuracy should therefore be read together; a combined measure, such as correct chains out of all 50, would make the comparison sharper (on that measure the conventional condition reaches about half the collaborative one).

4 Limitations

- **Constructed test set.** The 50 documents follow the internal template but are built from public material; real collections are larger, have more versions and stricter access rules.
- **Small evaluation.** 50 task chains, one round of testing. A future version should report the number of participants and how they were assigned to conditions.
- **System-only errors.** 81% accuracy alone means the system is not ready to answer without a person checking, particularly when versions conflict.
- **Untested extensions.** The graph and agent interfaces were not evaluated.

5 Contribution of the first author

Lintong Han co-designed the system, managed the program, and worked on retrieval over long documents and on evaluation. The system was built and tested by the team listed above.

References

[1] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[2] Y. Huang et al. LayoutLMv3: pre-training for document AI with unified text and image masking. *ACM Multimedia*, 2022.

[3] J. Herzig et al. TaPas: weakly supervised table parsing via pre-training. *ACL*, 2020.

[4] M. Mathew, D. Karatzas, C. V. Jawahar. DocVQA: a dataset for VQA on document images. *WACV*, 2021.

[5] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3(4), 2009.

[6] N. Thakur et al. BEIR: a heterogeneous benchmark for zero-shot evaluation of information retrieval models. *NeurIPS Datasets and Benchmarks*, 2021.

[7] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. *EMNLP*, 2020.

[8] J. Johnson, M. Douze, H. Jégou. Billion-scale similarity search with GPUs. arXiv:1702.08734, 2017.